

LABORATORIO 2

Código para la lectura de sensores (DHT11, MQ2, Humedad de Suelo) con ESP32 y envío de datos vía MQTT para integración en Node-RED:

```
#include <WiFi.h>
#include <PubSubClient.h>
#include <DHT.h>

#define DHTPIN 14
#define DHTTYPE DHT11
#define MQ2PIN 35
#define SUELOPIN 32

const char* ssid = "S24 FE";
const char* password = "hqv9598";
const char* mqtt_server = "broker.emqx.io";

WiFiClient espClient;
PubSubClient client(espClient);
DHT dht(DHTPIN, DHTTYPE);

void reconnectMQTT() {
    while (!client.connected()) {
        Serial.print("Intentando conectar a MQTT...");
        if (client.connect("ESP32_Client")) {
            Serial.println(" ¡Conectado!");
        } else {
            Serial.print(" Fallo (rc=");
            Serial.print(client.state());
            Serial.println("), reintentando en 5 segundos...");
            delay(5000);
        }
    }
}

void setup() {
    Serial.begin(115200);

    WiFi.begin(ssid, password);
    Serial.print("Conectando a WiFi...");
    while (WiFi.status() != WL_CONNECTED) {
        delay(500);
        Serial.print(".");
    }
    Serial.println("\nConectado a WiFi!");

    client.setServer(mqtt_server, 1883);
    reconnectMQTT();
    dht.begin();
}
```

```

}

void loop() {
  if (!client.connected()) {
    reconnectMQTT();
  }
  client.loop();

  float temperatura = dht.readTemperature();
  float humedad = dht.readHumidity();
  int gasValue = analogRead(MQ2PIN);
  int humedadSuelo = analogRead(SUELOPIN);

  Serial.print("Temperatura leída: ");
  Serial.println(temperatura);
  Serial.print("Humedad leída: ");
  Serial.println(humedad);

  if (isnan(temperatura) || isnan(humedad)) {
    Serial.println("⚠ Error: Sensor DHT11 no está enviando datos.");
  } else {
    char tempStr[6], humStr[6];
    dtostrf(temperatura, 2, 2, tempStr);
    dtostrf(humedad, 2, 2, humStr);

    if (client.publish("esp32/dht11/temperatura", tempStr)) {
      Serial.println("✅ Temperatura enviada correctamente.");
    } else {
      Serial.println("❌ Error al enviar Temperatura.");
    }

    if (client.publish("esp32/dht11/humedad", humStr)) {
      Serial.println("✅ Humedad enviada correctamente.");
    } else {
      Serial.println("❌ Error al enviar Humedad.");
    }
  }
}

char gasStr[6], sueloStr[6];
sprintf(gasStr, "%d", gasValue);
sprintf(sueloStr, "%d", humedadSuelo);

if (client.publish("esp32/mq2/gas", gasStr)) {
  Serial.println("✅ MQ2 Gas enviado correctamente.");
} else {
  Serial.println("❌ Error al enviar MQ2 Gas.");
}

if (client.publish("esp32/suelo/humedad", sueloStr)) {
  Serial.println("✅ Humedad del suelo enviada correctamente.");
}

```

```

} else {
  Serial.println("✗ Error al enviar humedad del suelo.");
}

Serial.print("Valor MQ2: ");
Serial.println(gasValue);
Serial.print("Humedad del suelo: ");
Serial.println(humedadSuelo);

delay(3000);
}

```

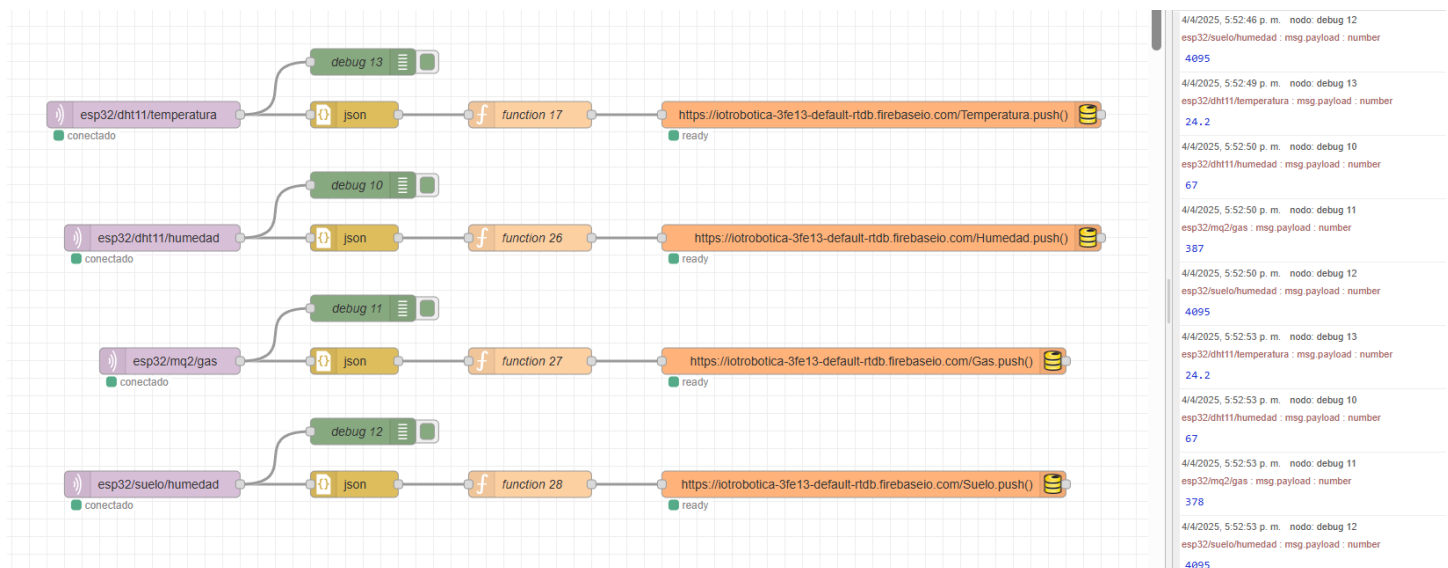
Captura de los datos enviándose correctamente:

```

19:46:31.617 -> ✓ Temperatura enviada correctamente.
19:46:31.617 -> ✓ Humedad enviada correctamente.
19:46:31.617 -> ✓ MQ2 Gas enviado correctamente.
19:46:31.708 -> ✓ Humedad del suelo enviada correctamente.
19:46:31.708 -> Valor MQ2: 527
19:46:31.708 -> Humedad del suelo: 4095
19:46:34.644 -> Temperatura leída: 26.00
19:46:34.680 -> Humedad leída: 62.00
19:46:34.680 -> ✓ Temperatura enviada correctamente.
19:46:34.680 -> ✓ Humedad enviada correctamente.
19:46:34.680 -> ✓ MQ2 Gas enviado correctamente.
19:46:34.680 -> ✓ Humedad del suelo enviada correctamente.
19:46:34.680 -> Valor MQ2: 503
19:46:34.680 -> Humedad del suelo: 4095
19:46:37.702 -> Temperatura leída: 26.00
19:46:37.702 -> Humedad leída: 62.00

```

Estructura de Nodos para la recepción de datos por MQTT desde ESP32 y almacenamiento automático en Firebase Realtime Database:

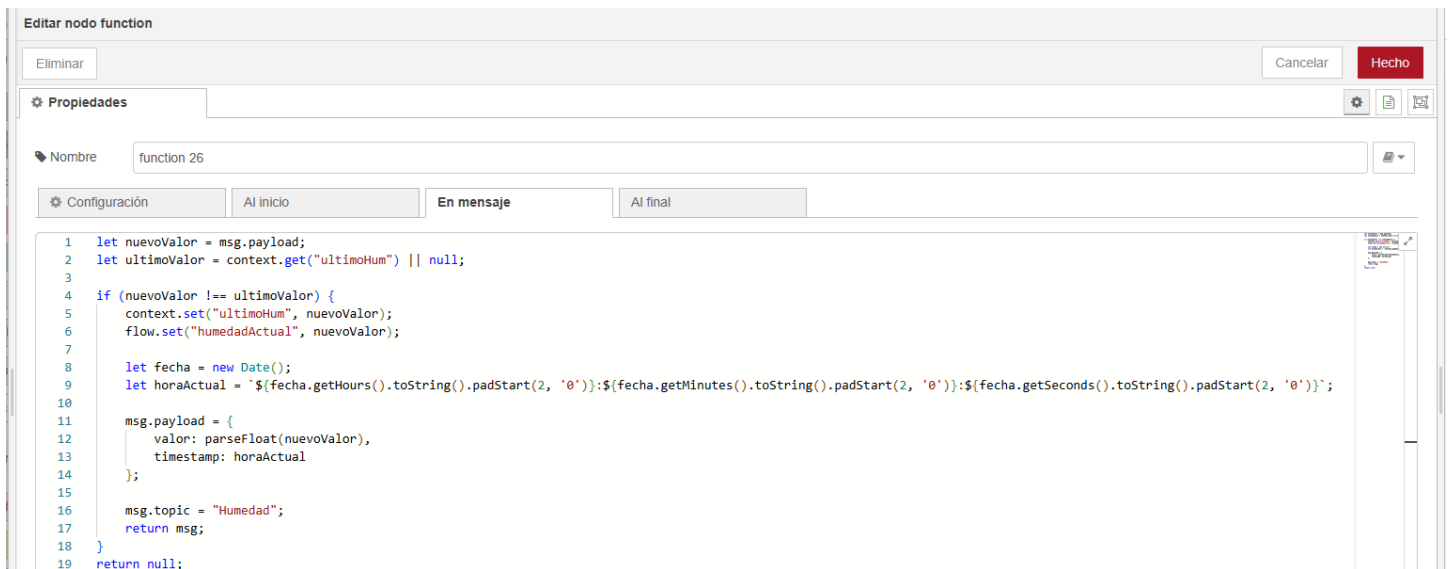


Función que envía el valor de temperatura solo si ha cambiado, con marca de tiempo incluida:



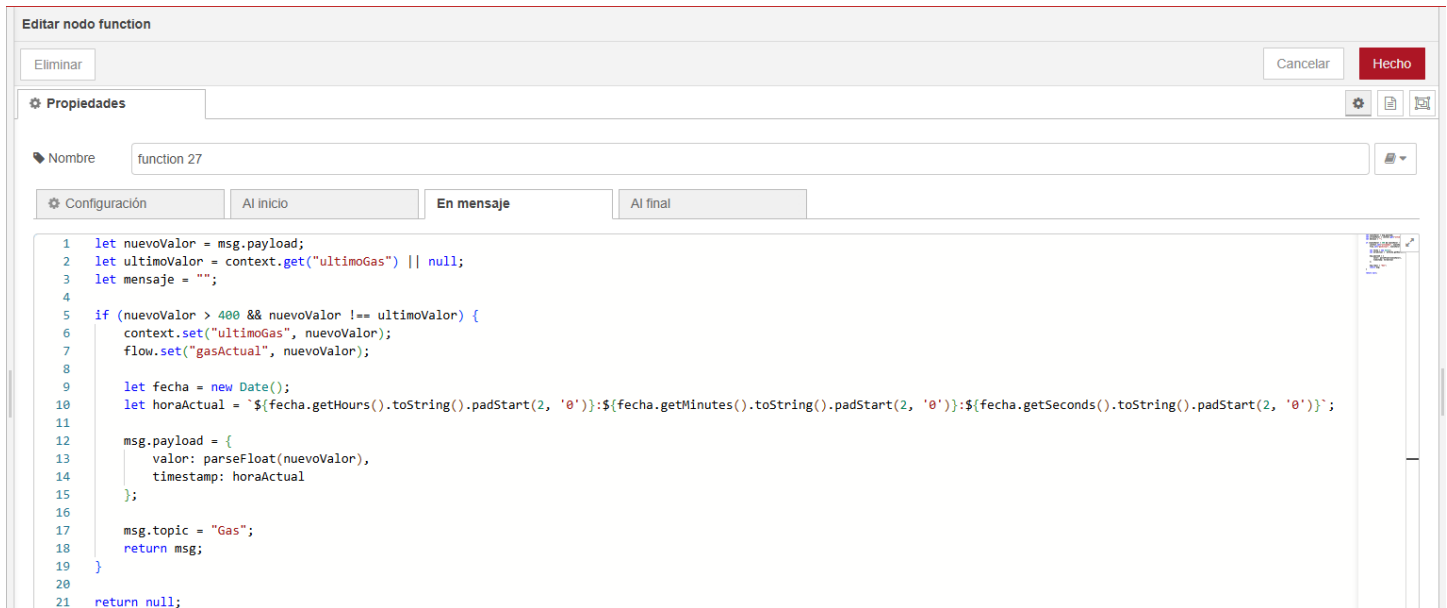
```
1 let nuevoValor = msg.payload;
2 let ultimoValor = context.get("ultimoValorTemperatura") || null;
3
4 if (nuevoValor !== ultimoValor) {
5   context.set("ultimoValorTemperatura", nuevoValor);
6   flow.set("valorActualTemperatura", nuevoValor);
7
8   let fecha = new Date();
9   let horaActual = `${fecha.getHours().toString().padStart(2, '0')}:${fecha.getMinutes().toString().padStart(2, '0')}:${fecha.getSeconds().toString().padStart(2, '0')}`;
10
11   msg.payload = {
12     valor: parseFloat(nuevoValor),
13     timestamp: horaActual
14   };
15
16   msg.topic = "Temperatura";
17   return msg;
18 }
19 return null;
```

Función que envía el valor de humedad solo si ha cambiado, con marca de tiempo incluida:



```
1 let nuevoValor = msg.payload;
2 let ultimoValor = context.get("ultimoHum") || null;
3
4 if (nuevoValor !== ultimoValor) {
5   context.set("ultimoHum", nuevoValor);
6   flow.set("humedadActual", nuevoValor);
7
8   let fecha = new Date();
9   let horaActual = `${fecha.getHours().toString().padStart(2, '0')}:${fecha.getMinutes().toString().padStart(2, '0')}:${fecha.getSeconds().toString().padStart(2, '0')}`;
10
11   msg.payload = {
12     valor: parseFloat(nuevoValor),
13     timestamp: horaActual
14   };
15
16   msg.topic = "Humedad";
17   return msg;
18 }
19 return null;
```

Función que envía el valor de gas solo si se detecta y si este ha cambiado, con marca de tiempo incluida:



Editar nodo function

Eliminar Cancelar Hecho

Propiedades

Nombre function 27

Configuración Al inicio En mensaje Al final

```
1 let nuevoValor = msg.payload;
2 let ultimoValor = context.get("ultimoGas") || null;
3 let mensaje = "";
4
5 if (nuevoValor > 400 && nuevoValor !== ultimoValor) {
6   context.set("ultimoGas", nuevoValor);
7   flow.set("gasActual", nuevoValor);
8
9   let fecha = new Date();
10  let horaActual = `${fecha.getHours().toString().padStart(2, '0')}:${fecha.getMinutes().toString().padStart(2, '0')}:${fecha.getSeconds().toString().padStart(2, '0')}`;
11
12  msg.payload = {
13    valor: parseFloat(nuevoValor),
14    timestamp: horaActual
15  };
16
17  msg.topic = "Gas";
18  return msg;
19 }
20
21 return null;
```

Función que envía el valor de humedad de suelo solo si se detecta y si este ha cambiado, con marca de tiempo incluida:



Editar nodo function

Eliminar Cancelar Hecho

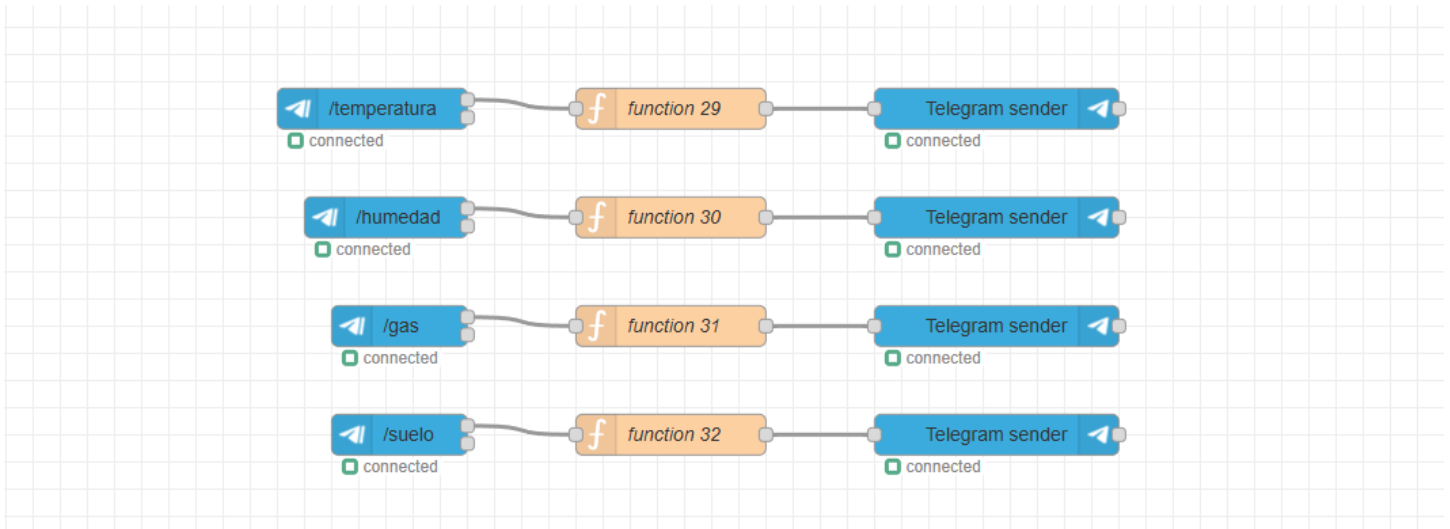
Propiedades

Nombre function 28

Configuración Al inicio En mensaje Al final

```
1 let nuevoValor = msg.payload;
2 let ultimoValor = context.get("ultimoSue") || null;
3
4 if (nuevoValor >= 1000 && nuevoValor <= 2800 && nuevoValor !== ultimoValor) {
5   context.set("ultimoSue", nuevoValor);
6   flow.set("humedadSueloActual", nuevoValor);
7
8   let fecha = new Date();
9   let horaActual = `${fecha.getHours().toString().padStart(2, '0')}:${fecha.getMinutes().toString().padStart(2, '0')}:${fecha.getSeconds().toString().padStart(2, '0')}`;
10
11  msg.payload = {
12    valor: parseFloat(nuevoValor),
13    timestamp: horaActual
14  };
15
16  msg.topic = "Suelo";
17  return msg;
18 }
19
20 return null;
```

Estructura de nodos para enviar los datos por Telegram mediante comandos:



Función para generar el mensaje con la temperatura recibida y la hora actual:

```
Editar nodo function
Eliminar Cancelar Hecho

Propiedades
Nombre function 29

Configuración AI inicio En mensaje AI final

1 let temperatura = flow.get("valorActualTemperatura") || "No disponible";
2
3 let fecha = new Date();
4 let horaActual = `${fecha.getHours()}:${fecha.getMinutes()}:${fecha.getSeconds()}`;
5
6 let mensaje = `Temperatura: ${temperatura} %\n🕒 Hora actual: ${horaActual}`;
7
8 msg.payload = {
9   "chatId": 6682379096,
10  "type": "message",
11  "content": mensaje
12 };
13
14 return msg;
```

Función para generar el mensaje con la humedad recibida y la hora actual:

```
Editar nodo function
Eliminar Cancelar Hecho

Propiedades
Nombre function 30

Configuración AI inicio En mensaje AI final

1 let humedad = flow.get("humedadActual") || "No disponible";
2
3 let fecha = new Date();
4 let horaActual = `${fecha.getHours()}:${fecha.getMinutes()}:${fecha.getSeconds()}`;
5
6 let mensaje = `💧 Humedad: ${humedad} %\n🕒 Hora actual: ${horaActual}`;
7
8 msg.payload = {
9   "chatId": 6682379096,
10  "type": "message",
11  "content": mensaje
12 };
13
14 return msg;
```

Función para generar el mensaje con el gas recibido y la hora actual:

Editar nodo function

Eliminar Cancelar Hecho

Propiedades

Nombre function 31

Configuración AI inicio En mensaje AI final

```
1 let gas = flow.get("gasActual") || 0;
2
3 let fecha = new Date();
4 let horaActual = `${fecha.getHours()}:${fecha.getMinutes()}:${fecha.getSeconds()}`;
5
6 let mensaje;
7
8 if (gas > 400) {
9   mensaje = `🔴 ¡Alerta! Gas detectado: ${gas} ppm\n🕒 Hora actual: ${horaActual}`;
10 } else {
11   mensaje = `✅ No se detectó gas.\n🕒 Hora actual: ${horaActual}`;
12 }
13
14 msg.payload = {
15   "chatId": 6682379096,
16   "type": "message",
17   "content": mensaje
18 };
19
20 return msg;
```

Función para generar el mensaje con la humedad de suelo recibida y la hora actual:

Editar nodo function

Eliminar Cancelar Hecho

Propiedades

Nombre function 32

Configuración AI inicio En mensaje AI final

```
1 let humedadSuelo = flow.get("humedadSueloActual") || 0;
2
3 let fecha = new Date();
4 let horaActual = `${fecha.getHours()}:${fecha.getMinutes()}:${fecha.getSeconds()}`;
5
6 let mensaje;
7
8 if (humedadSuelo >= 1000 && humedadSuelo <= 1500) {
9   mensaje = `💧 Suelo húmedo: ${humedadSuelo}\n🕒 Hora actual: ${horaActual}`;
10 } else if (humedadSuelo > 1500 && humedadSuelo <= 2000) {
11   mensaje = `🌱 Tierra semihúmeda: ${humedadSuelo}\n🕒 Hora actual: ${horaActual}`;
12 } else if (humedadSuelo > 2000 && humedadSuelo <= 2800) {
13   mensaje = `🌵 Tierra seca: ${humedadSuelo}\n🕒 Hora actual: ${horaActual}`;
14 } else {
15   mensaje = `✅ No se detectó humedad en el suelo.\n🕒 Hora actual: ${humedadSuelo}`;
16 }
17
18 msg.payload = {
19   "chatId": 6682379096,
20   "type": "message",
21   "content": mensaje
22 };
23
24 return msg;
```

Captura mostrando los mensajes recibidos en Telegram:



Código para la visualización en tiempo real de datos de sensores con Firebase utilizando gráficos:

```
<!DOCTYPE html>
<html>

<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Datos de Sensores</title>

  <script defer src="/_/firebase/11.6.0/firebase-app-compat.js"></script>
  <script defer src="/_/firebase/11.6.0/firebase-database-compat.js"></script>
  <script defer src="/_/firebase/init.js?useEmulator=true"></script>
  <script src="https://cdn.jsdelivr.net/npm/chart.js"></script>

<style>
  * {
    box-sizing: border-box;
    margin: 0;
    padding: 0;
  }

  body {
    background-color: #121212;
    color: #f1f1f1;
    font-family: "Segoe UI", Tahoma, Geneva, Verdana, sans-serif;
    padding: 20px;
    text-align: center;
  }

  h1 {
    color: #6ee7ff;
    margin-bottom: 30px;
  }

  .sensor-box {
    background: #1e1e1e;
    border-radius: 12px;
    padding: 20px;
    margin: 20px auto;
    width: 100%;
    max-width: 650px;
    box-shadow: 0 0 10px rgba(43, 227, 252, 0.636);
    transition: transform 0.3s ease;
  }

  .sensor-box:hover {
    transform: translateY(-4px);
    box-shadow: 0 0 15px rgba(101, 237, 255, 0.723);
  }

  .sensor-box h2 {
    margin-bottom: 10px;
    color: #ffffff;
  }

  .sensor-value {
    font-size: 24px;
    font-weight: bold;
    color: #6ee7ff;
    margin-bottom: 10px;
  }

  .timestamp {
    font-size: 14px;
    color: #aaaaaa;
  }
```

```

#load {
  margin-top: 30px;
  color: #888;
}

@media (max-width: 600px) {
  .sensor-box {
    width: 90%;
  }

  .sensor-value {
    font-size: 22px;
  }

  h1 {
    font-size: 24px;
  }
}
</style>
</head>

<body>
  <h1>Datos en Tiempo Real</h1>

  <div class="sensor-box">
    <canvas id="temp-chart" height="150"></canvas>
    <h2>Temperatura</h2>
    <div class="sensor-value" id="temp">Cargando...</div>
    <div class="timestamp" id="temp-time"></div>
  </div>

  <div class="sensor-box">
    <canvas id="hum-chart" height="150"></canvas>
    <h2>Humedad</h2>
    <div class="sensor-value" id="hum">Cargando...</div>
    <div class="timestamp" id="hum-time"></div>
  </div>

  <div class="sensor-box">
    <canvas id="gas-chart" height="150"></canvas>
    <h2>Gas</h2>
    <div class="sensor-value" id="gas">Cargando...</div>
    <div class="timestamp" id="gas-time"></div>
  </div>

  <div class="sensor-box">
    <canvas id="soil-chart" height="150"></canvas>
    <h2>Humedad de Suelo</h2>
    <div class="sensor-value" id="soil">Cargando...</div>
    <div class="timestamp" id="soil-time"></div>
  </div>

  <p id="load">Conectando a Firebase...</p>

  <script>
    document.addEventListener("DOMContentLoaded", function () {
      const db = firebase.database();
      const loadEl = document.getElementById("load");

      const sensores = {
        "Temperatura": {
          valueEl: document.getElementById("temp"),
          timeEl: document.getElementById("temp-time"),
          chartEl: document.getElementById("temp-chart"),
          data: [],
          labels: [],
        },
        "Humedad": {

```

```

    valueEl: document.getElementById("hum"),
    timeEl: document.getElementById("hum-time"),
    chartEl: document.getElementById("hum-chart"),
    data: [],
    labels: [],
  },
  "Gas": {
    valueEl: document.getElementById("gas"),
    timeEl: document.getElementById("gas-time"),
    chartEl: document.getElementById("gas-chart"),
    data: [],
    labels: [],
  },
  "Suelo": {
    valueEl: document.getElementById("soil"),
    timeEl: document.getElementById("soil-time"),
    chartEl: document.getElementById("soil-chart"),
    data: [],
    labels: [],
  },
};

const charts = {};

Object.keys(sensores).forEach(sensor => {
  const ctx = sensores[sensor].chartEl.getContext("2d");
  charts[sensor] = new Chart(ctx, {
    type: "line",
    data: {
      labels: sensores[sensor].labels,
      datasets: [{
        label: sensor,
        data: sensores[sensor].data,
        borderColor: "#6ee7ff",
        backgroundColor: "rgba(110, 231, 255, 0.2)",
        tension: 0.3
      }]
    },
    options: {
      responsive: true,
      scales: {
        x: { display: true, title: { display: true, text: "Hora" } },
        y: { beginAtZero: true }
      },
      plugins: {
        legend: { labels: { color: "#f1f1f1" } }
      }
    }
  });

  const sensorRef = db.ref(sensor).limitToLast(20);

  sensorRef.on("child_added", snapshot => {
    const data = snapshot.val();
    if (data) {
      loadEl.style.display = "none";

      const sensorData = sensores[sensor];
      let hora = data.timestamp;

      sensorData.valueEl.textContent = data.valor;
      sensorData.timeEl.textContent = "Última actualización: " + hora;

      sensorData.labels.push(hora);
      sensorData.data.push(data.valor);

      if (sensorData.data.length > 20) {
        sensorData.labels.shift();

```

```

        sensorData.data.shift();
    }

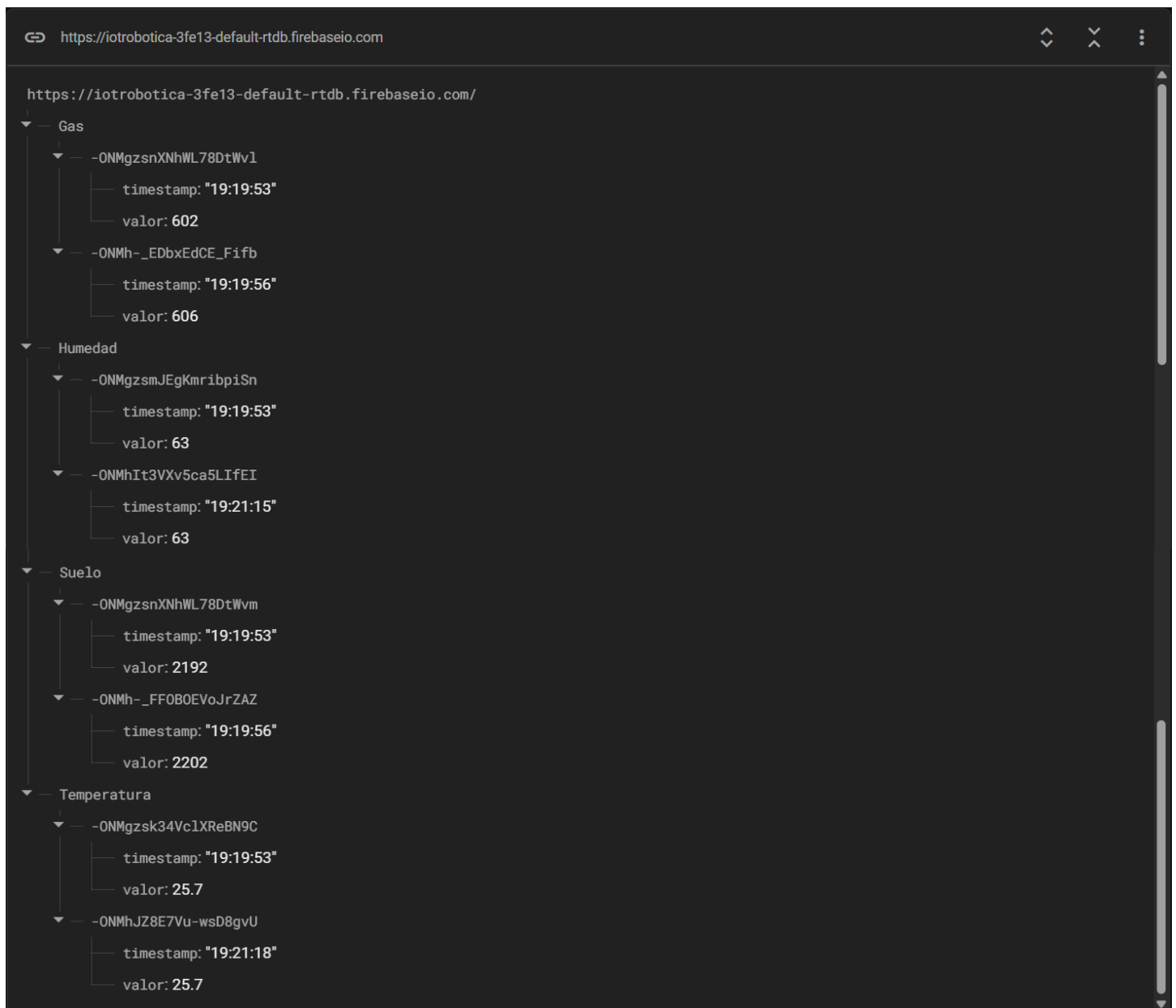
    charts[sensor].update();
}
});
});
});
</script>

</body>

</html>

```

Visualización de los datos en Firebase Realtime Database:



The screenshot shows the Firebase Realtime Database console for the database `https://iotrobotica-3fe13-default-rtdb.firebaseio.com/`. The data is organized into a tree structure with four main categories: Gas, Humedad, Suelo, and Temperatura. Each category contains two nodes, each with a timestamp and a value.

Categoría	Identificador	timestamp	valor
Gas	-ONMgzsnXNhWL78DtWv1	"19:19:53"	602
	-ONMh-_EDbxEdCE_Fifb	"19:19:56"	606
Humedad	-ONMgzsmJEgKmribpiSn	"19:19:53"	63
	-ONMhIt3VXv5ca5LIfeI	"19:21:15"	63
Suelo	-ONMgzsnXNhWL78DtWvm	"19:19:53"	2192
	-ONMh-_FF0B0EVoJrZAZ	"19:19:56"	2202
Temperatura	-ONMgzsk34Vc1XReBN9C	"19:19:53"	25.7
	-ONMhJZ8E7Vu-wsD8gvU	"19:21:18"	25.7

Enlace de la web:

<https://iotrobotica-3fe13.firebaseio.com/>

Visualización de la web para el monitoreo en tiempo real de sensores: temperatura, humedad, gas y humedad del suelo:

